

HTML Basics

Chapter 1: Introduction to HTML

- **1.1 What is HTML?**
 - Understanding the role of HTML in web development.
- **1.2 Setting Up Your Environment**
 - Choosing and installing a code editor (e.g., Visual Studio Code, Sublime Text).

Chapter 2: Basic HTML Structure

- **2.1 Creating Your First HTML Document**
 - Basic syntax and structure of an HTML file.
- **2.2 Understanding HTML Tags and Elements**
 - What are tags? Difference between opening and closing tags.

Chapter 3: Text Elements

- **3.1 Headings and Paragraphs**
 - Using `<h1>` to `<h6>` for headings and `<p>` for paragraphs.
- **3.2 Formatting Text**
 - Bold, italic, underline, and other text formatting options.

Chapter 4: Lists in HTML

- **4.1 Unordered and Ordered Lists**
 - Creating bullet points with `<ul>` and numbered lists with `<ol>`.
- **4.2 Description Lists**
 - Using `<dl>`, `<dt>`, and `<dd>` for definition lists.

Chapter 5: Links and Navigation

- **5.1 Creating Links**
 - Using the `<a>` tag to create hyperlinks.
- **5.2 Navigation Bars**
 - Structuring navigation using lists and links.

Chapter 6: Images and Multimedia

- **6.1 Adding Images**
 - Using the `<img>` tag, understanding attributes like `src` and `alt`.
- **6.2 Embedding Videos**
 - Introduction to embedding multimedia content.

Chapter 7: Tables

- **7.1 Creating Tables**
 - Using `<table>`, `<tr>`, `<th>`, and `<td>` to structure tabular data.
- **7.2 Table Attributes**
 - Understanding attributes like `colspan` and `rowspan`.

Chapter 8: Forms and User Input

- **8.1 Creating Forms**
 - Using the `<form>` tag to collect user input.
- **8.2 Input Types**
 - Different types of inputs (text, radio buttons, checkboxes).

Chapter 9: Semantic HTML

- **9.1 Understanding Semantic Elements**
 - Importance of semantic tags like `<header>`, `<footer>`, `<article>`, etc.

Chapter 10: Responsive Design Basics

- **10.1 Introduction to Responsive Web Design**
 - Basic principles of making websites responsive using media queries.

Chapter 11: Practical Project

- **11.1 Building a Simple Web Page**
 - Applying what you've learned by creating a complete web page from scratch.

Chapter 1: Introduction to HTML

- **1.1 What is HTML?**
 - Understanding the role of HTML in web development.
- **1.2 Setting Up Your Environment**
 - Choosing and installing a code editor (e.g., Visual Studio Code, Sublime Text).

1.1 What is HTML?

HTML (HyperText Markup Language) is the standard language used to create and structure content on the web. It defines the structure and layout of web pages by utilizing a system of tags that enclose various content elements, such as headings, paragraphs, images, links, and forms.

Key Points:

- HTML provides the foundation for all web pages. It is responsible for organizing content on the page.
- **Markup Language:** HTML is a markup language, meaning it uses tags to annotate content, indicating its type (e.g., text, image, link) and structure (e.g., headings, paragraphs, lists).

- **Web Browsers:** Web browsers, such as Google Chrome, Firefox, and Safari, interpret HTML code and render it into visually accessible web pages.
- HTML, while essential for structure, is often combined with **CSS (Cascading Style Sheets)** for styling and **JavaScript** for interactivity.

Example of a Simple HTML Page:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Introduction to HTML</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <p>This is a simple HTML document that showcases basic structure.</p>
</body>
</html>
```

Important HTML Elements:

- `<html>`: The root element of an HTML document.
- `<head>`: Contains meta-information such as title, character encoding, etc.
- `<body>`: Contains the content that is displayed to the user.
- `<h1>` - `<h6>`: Headings for titles and subtitles.
- `<p>`: Paragraph for text content.

1.2 Setting Up Your Environment

To begin writing and experimenting with HTML, you'll need to set up a **code editor**. Code editors are tools that allow you to write, edit, and manage your HTML

code efficiently.

Choosing a Code Editor:

Some popular code editors include:

1. Visual Studio Code (VS Code):

- **Description:** VS Code is a lightweight, powerful code editor developed by Microsoft. It is highly customizable and supports a range of extensions, including HTML, CSS, JavaScript, and many others.
- **Features:**
 - Syntax highlighting for HTML.
 - Extensions like Live Server for previewing web pages.
 - Integrated Git support.
- **Installation:** Available for Windows, Mac, and Linux. You can download it from [here](#).

2. Sublime Text:

- **Description:** Sublime Text is a fast and lightweight text editor. It is simple, clean, and highly efficient.
- **Features:**
 - Fast and responsive.
 - Supports multiple cursors and quick editing.
 - Syntax highlighting and auto-completion for HTML.
- **Installation:** Available for Windows, Mac, and Linux. You can download it from [here](#).

3. Notepad++:

- **Description:** Notepad++ is a free source code editor for Windows that supports many programming languages, including HTML.
- **Features:**
 - Lightweight and easy to use.

- Syntax highlighting for HTML.
- **Installation:** Available for Windows. You can download it from [here](#).

Installing Visual Studio Code (VS Code):

1. Go to the [VS Code download page](#).
2. Choose the appropriate version for your operating system (Windows, Mac, Linux).
3. Follow the installation instructions.

Creating Your First HTML File:

1. Open your code editor (e.g., Visual Studio Code).
2. Create a new file with the extension `.html` (e.g., `index.html`).
3. Write your HTML code in the file.
4. Save the file.
5. Open the HTML file in a web browser to see the result.

Previewing HTML Code:

- You can use extensions like **Live Server** in Visual Studio Code to instantly preview your HTML files in the browser without manually refreshing them. This tool automatically reloads your browser whenever you save changes to your code.

Summary of Chapter 1:

- HTML is the foundation of web development, used to create and structure web content.
- You need a code editor to write and manage your HTML code, and options like Visual Studio Code, Sublime Text, or Notepad++ are great choices.
- After setting up your environment, creating an HTML file and previewing it in a web browser is simple and easy.

References:

- [HTML Overview by Mozilla Developer Network \(MDN\)](#)
- [Visual Studio Code Documentation](#)
- [Sublime Text Documentation](#)

Chapter 2: Basic HTML Structure

- **2.1 Creating Your First HTML Document**
 - Basic syntax and structure of an HTML file.
- **2.2 Understanding HTML Tags and Elements**
 - What are tags? Difference between opening and closing tags.

2.1 Creating Your First HTML Document

When you start creating an HTML document, the most important thing is to understand its basic structure. HTML files are text files that describe the structure and content of a webpage.

Basic HTML Syntax:

An HTML document follows a specific structure that allows browsers to understand and render the content correctly. Here's a simple example of a basic HTML document:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Webpage</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first HTML document.</p>
```

```
</body>
</html>
```

Explanation of the Structure:

1. `<!DOCTYPE html>` :
 - The `DOCTYPE` declaration defines the document type and version of HTML. It ensures that the document is rendered in standards mode, which makes the browser display the page according to modern web standards. For HTML5, this declaration is simply `<!DOCTYPE html>`.
2. `<html>` :
 - This is the root element of the HTML document. It encapsulates all the content of the web page.
3. `<head>` :
 - The `<head>` element contains meta-information about the document, such as the title, character encoding, and viewport settings. This information is not visible on the webpage but is important for web browsers and search engines.
4. `<meta charset="UTF-8">` :
 - This tag specifies the character encoding for the document, ensuring that text, including special characters, is displayed correctly.
5. `<meta name="viewport" content="width=device-width, initial-scale=1.0">` :
 - This tag is used for responsive design. It ensures the page looks good on all screen sizes (important for mobile devices).
6. `<title>` :
 - The content of the `<title>` tag appears as the title of the webpage in the browser's title bar or tab.
7. `<body>` :
 - The `<body>` element contains the actual content that will be displayed on the webpage. This is where you'll put your text, images, links, and other

content.

8. `<h1>` :

- The `<h1>` element represents the main heading of the page. It's typically the most important heading and often appears in large text.

9. `<p>` :

- The `<p>` tag defines a paragraph. In this case, it contains the text "This is my first HTML document."

2.2 Understanding HTML Tags and Elements

HTML uses **tags** to structure content on a webpage. Tags are enclosed in angle brackets (`< >`), and most tags come in pairs: an opening tag and a closing tag. Let's dive deeper into these concepts:

What Are HTML Tags?

- **HTML Tags** are the building blocks of an HTML document. They are used to define elements within the page and describe the content's structure and function.
- Tags typically come in pairs:
 - **Opening tag**: Marks the start of an HTML element (e.g., `<p>`).
 - **Closing tag**: Marks the end of an HTML element (e.g., `</p>`). The only difference is that the closing tag contains a forward slash `/`.

Opening and Closing Tags:

In HTML, many elements require both an opening and a closing tag.

- **Opening tag**: `<tag>`
 - This indicates the start of an HTML element.
- **Closing tag**: `</tag>`
 - This indicates the end of the element.

For example, a **paragraph** in HTML is enclosed in `<p>` (opening) and `</p>` (closing) tags.

```
<p>This is a paragraph.</p>
```

Self-closing Tags:

Some HTML elements, such as images and line breaks, don't require a closing tag. These elements are called **self-closing tags**. For example:

- `<img>`: Used to embed an image. It doesn't require a closing tag, but it typically includes the `src` (source) attribute to specify the image path.

```

```

- `<br>`: Used to insert a line break. It also doesn't require a closing tag.

```
<p>This is the first line.<br>This is the second line.</p>
```

Attributes in HTML Tags:

HTML tags can also contain **attributes**, which provide additional information about an element. Attributes are written within the opening tag and are always in the form of key-value pairs.

- **Example of an anchor tag (`<a>`) with an attribute:**

```
<a href="https://www.example.com" target="_blank">Visit Example</a>
```

- `href="https://www.example.com"` is an attribute that specifies the URL the link points to.
- `target="_blank"` is an attribute that makes the link open in a new browser tab.
- **Example of an image tag (`<img>`) with attributes:**

```

```

- `src="image.jpg"` specifies the image source.
 - `alt="A beautiful scenery"` provides alternative text if the image doesn't load.
 - `width="500"` and `height="300"` set the dimensions of the image.
-

Summary of Chapter 2:

- **Creating Your First HTML Document:** The basic structure of an HTML document includes the `DOCTYPE`, `html`, `head`, and `body` tags, along with elements like `<title>`, `<h1>`, and `<p>`.
 - **HTML Tags and Elements:** HTML tags define the structure of a webpage. Tags are enclosed in angle brackets, with most elements having both opening and closing tags. Self-closing tags like `<img>` and `<br>` don't need closing tags. Tags can also have attributes that provide additional information.
-

References:

- [HTML Tags - Mozilla Developer Network \(MDN\)](#)
- [HTML Syntax - W3Schools](#)

Chapter 3: Text Elements

- **3.1 Headings and Paragraphs**
 - Using `<h1>` to `<h6>` for headings and `<p>` for paragraphs.
- **3.2 Formatting Text**
 - Bold, italic, underline, and other text formatting options.

3.1 Headings and Paragraphs

Headings and paragraphs are fundamental components of text content in HTML. They allow you to structure your content hierarchically and present it in a readable way.

Using `<h1>` to `<h6>` for Headings

HTML provides six levels of headings, from `<h1>` (most important) to `<h6>` (least important). Headings are used to define titles or sections of content, helping organize the page and improve accessibility.

Example:

```
<h1>This is a Heading 1 (Main Title)</h1>
<h2>This is a Heading 2 (Subheading)</h2>
<h3>This is a Heading 3 (Subsection)</h3>
<h4>This is a Heading 4</h4>
<h5>This is a Heading 5</h5>
<h6>This is a Heading 6</h6>
```

- `<h1>`: Typically used for the main title or the most important heading on the page (e.g., the page title).
- `<h2>` to `<h6>`: Used for subsections or smaller headings, with `<h2>` being the second most important and `<h6>` the least important.

Why Use Headings?

- **SEO (Search Engine Optimization)**: Search engines prioritize headings (especially `<h1>`) to understand the main topic of the page.
- **Accessibility**: Screen readers use headings to help visually impaired users navigate content efficiently.
- **Content Organization**: Headings make content easier to scan and understand for all users.

Using `<p>` for Paragraphs

The `<p>` tag is used to define paragraphs of text in HTML. It automatically creates space before and after the paragraph for better readability.

Example:

```
<p>This is a paragraph of text. Paragraphs help organize the
```

content on a webpage and make it easier to read.</p>

- A **paragraph** is a block of text that is typically separated from other blocks of text by spacing.
- You can write multiple paragraphs on the same page, and each paragraph should be enclosed by `<p>` tags.

3.2 Formatting Text

HTML provides various tags to format text for emphasis or style. These tags allow you to make text **bold**, **italic**, **underlined**, and more.

Bold Text: `<b>` and `<strong>`

- `<b>` : This tag makes text **bold** for visual emphasis.
- `<strong>` : This tag also makes text bold but with a semantic meaning, indicating that the text is of strong importance. It is better for accessibility and search engines.

Example:

```
<p>This is <b>bold</b> text.</p>
<p>This is <strong>strong</strong> text.</p>
```

- `<b>` is purely for visual effect, while `<strong>` has a semantic meaning, indicating emphasis or importance.

Italic Text: `<i>` and `<em>`

- `<i>` : This tag italicizes text for visual emphasis.
- `<em>` : This tag italicizes text but with a semantic meaning, indicating that the text should be emphasized. It is important for accessibility and search engines.

Example:

```
<p>This is <i>italic</i> text.</p>
<p>This is <em>emphasized</em> text.</p>
```

- `<i>` is typically used for visual style (e.g., for foreign words), while `<em>` indicates that the text is important or should be stressed.

Underlined Text: `<u>`

The `<u>` tag is used to underline text. However, it is not as commonly used for general emphasis due to its historical association with hyperlinks. It's more often used for specific cases, such as for marking text that should stand out.

Example:

```
<p>This is <u>underlined</u> text.</p>
```

Strikethrough Text: `<s>`

The `<s>` tag is used to display text with a strikethrough, often used to indicate something that is no longer relevant or has been removed.

Example:

```
<p>This is <s>strikethrough</s> text.</p>
```

Subscript and Superscript: `<sub>` and `<sup>`

- `<sub>`: Used to display text as a subscript (e.g., in chemical formulas).
- `<sup>`: Used to display text as a superscript (e.g., in mathematical expressions).

Example:

```
<p>This is H<sub>2</sub>O (Water).</p>
<p>This is x<sup>2</sup> (x squared).</p>
```

- **Subscript:** Used for elements like H₂O.
- **Superscript:** Used for elements like x².

Monospaced Text: `<code>`

The `<code>` tag is used to display code snippets in a monospaced font, making it easier to read and distinguish from regular text.

Example:

```
<p>The command <code>git status</code> is used to check the s  
tatus of a Git repository.</p>
```

Line Break: `<br>`

The `<br>` tag is used to insert a line break within text. It doesn't require a closing tag and is typically used when you want to break a line but not start a new paragraph.

Example:

```
<p>This is line one.<br>This is line two.</p>
```

- The `<br>` tag forces a break in the text, placing the following text on a new line.

Preformatted Text: `<pre>`

The `<pre>` tag is used to display preformatted text, which retains spaces, line breaks, and formatting exactly as typed in the code.

Example:

```
<pre>  
  This is preformatted text.  
  It preserves spaces and line breaks.  
</pre>
```

- Useful for displaying code or poetry where the exact spacing and format matter.
-

Summary of Chapter 3:

- **Headings and Paragraphs:** Use `<h1>` to `<h6>` for headings (hierarchical titles) and `<p>` for paragraphs of text. Properly structured text enhances content accessibility and readability.
 - **Text Formatting:** HTML offers various tags for text formatting:
 - **Bold:** `<b>` for visual emphasis, `<strong>` for importance.
 - **Italic:** `<i>` for visual style, `<em>` for emphasis.
 - **Underline:** `<u>` for underlined text.
 - **Strikethrough:** `<s>` for crossed-out text.
 - **Subscript/Superscript:** `<sub>` and `<sup>` for formatting text above or below the normal line.
 - **Monospaced:** `<code>` for displaying code.
 - **Line Break:** `<br>` for breaking lines without starting a new paragraph.
 - **Preformatted Text:** `<pre>` for displaying text with preserved formatting.
-

References:

- [HTML Text Formatting - Mozilla Developer Network \(MDN\)](#).
- [HTML Text Elements - W3Schools](#)

Chapter 4: Lists in HTML

- **4.1 Unordered and Ordered Lists**
 - Creating bullet points with `<ul>` and numbered lists with `<ol>`.
- **4.2 Description Lists**
 - Using `<dl>`, `<dt>`, and `<dd>` for definition lists.

4.1 Unordered and Ordered Lists

HTML provides three types of lists to organize content: **unordered lists**, **ordered lists**, and **description lists** (covered in the next section). Lists are a great way to display multiple items in a structured format. The most commonly used lists are **unordered** and **ordered**.

Unordered Lists: `<ul>`

An **unordered list** is used when the order of the list items doesn't matter. It typically displays items with bullet points.

Example:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Orange</li>
</ul>
```

- `<ul>`: Defines an unordered list.
- `<li>`: Defines each list item.

The list above will appear as:

- Apple
- Banana
- Orange

You can also style the bullet points using CSS, or change the list style using the `list-style-type` property.

Ordered Lists: `<ol>`

An **ordered list** is used when the order of the list items is important. It typically displays items with numbers or letters.

Example:

```
<ol>
  <li>First step</li>
  <li>Second step</li>
  <li>Third step</li>
</ol>
```

- `<ol>` : Defines an ordered (numbered) list.
- `<li>` : Defines each list item.

The list above will appear as:

1. First step
2. Second step
3. Third step

You can change the numbering style (e.g., Roman numerals, letters) using the `type` attribute in the `<ol>` tag.

Example with custom numbering:

```
<ol type="a">
  <li>Step A</li>
  <li>Step B</li>
  <li>Step C</li>
</ol>
```

This will create a list with lowercase letters (a, b, c, etc.) as numbers.

Nesting Lists

You can also **nest** lists inside one another. This means adding an unordered or ordered list inside a list item.

Example of Nested Lists:

```
<ol>
  <li>Step 1</li>
```

```
<li>Step 2
  <ul>
    <li>Substep 1</li>
    <li>Substep 2</li>
  </ul>
</li>
<li>Step 3</li>
</ol>
```

This will display:

1. Step 1
 2. Step 2
 - Substep 1
 - Substep 2
 3. Step 3
-

4.2 Description Lists

A **description list** is used to define terms and their corresponding descriptions. This type of list is particularly useful when you want to provide additional information about a particular term or item.

Creating a Description List:

To create a description list, you use the following tags:

- `<dl>`: Defines a description list.
- `<dt>`: Defines a term (description list term).
- `<dd>`: Defines the description or definition of the term.

Example:

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language, used to create web pages.
```

```

</dd>

    <dt>CSS</dt>
    <dd>Cascading Style Sheets, used to style HTML elements.
</dd>

    <dt>JavaScript</dt>
    <dd>A programming language used to add interactivity to web pages.</dd>
</dl>

```

This will display:

- **HTML:** HyperText Markup Language, used to create web pages.
- **CSS:** Cascading Style Sheets, used to style HTML elements.
- **JavaScript:** A programming language used to add interactivity to web pages.

Nesting Description Lists

You can also nest description lists inside other lists. This can be useful for more complex information.

Example of Nested Description Lists:

```

<dl>
  <dt>Programming Languages</dt>
  <dd>Languages used to create software and applications.</dd>
  <dl>
    <dt>Python</dt>
    <dd>A popular programming language known for its simplicity.</dd>
    <dt>JavaScript</dt>
    <dd>A programming language primarily used for web development.</dd>
  </dl>
</dl>

```

```
</dl>  
</dl>
```

This will display:

- **Programming Languages:** Languages used to create software and applications.
 - **Python:** A popular programming language known for its simplicity.
 - **JavaScript:** A programming language primarily used for web development.
-

Summary of Chapter 4: Lists in HTML

- **Unordered Lists (`<ul>`):** Use unordered lists when the order of the items doesn't matter. They display items with bullet points.
 - **Ordered Lists (`<ol>`):** Use ordered lists when the order of the items is important. They display items with numbers or letters.
 - **Nesting Lists:** Lists can be nested within other lists, allowing for more complex structures.
 - **Description Lists (`<dl>`):** Used to define terms and their definitions. They are great for glossaries, dictionaries, or any content that requires a term-definition structure.
-

References:

- [HTML Lists - Mozilla Developer Network \(MDN\)](#)
- [HTML Lists - W3Schools](#)

Chapter 5: Links and Navigation

- **5.1 Creating Links**
 - Using the `<a>` tag to create hyperlinks.
- **5.2 Navigation Bars**
 - Structuring navigation using lists and links.

5.1 Creating Links

Hyperlinks (or simply "links") are one of the most important features of the web. Links allow users to navigate between different pages, websites, and resources. In HTML, the `<a>` (anchor) tag is used to create links.

Basic Link Structure with the `<a>` Tag

The anchor tag `<a>` is used to define a link, and it requires the `href` (hyperlink reference) attribute to specify the destination of the link. The content between the opening `<a>` and closing `</a>` tags is the clickable text or element.

Example of a Basic Link:

```
<a href="https://www.example.com">Click here to visit Example</a>
```

- `<a>`: The anchor tag that defines the hyperlink.
- `href="https://www.example.com"`: The `href` attribute contains the URL (Uniform Resource Locator) to which the link will navigate.
- **Link text:** The clickable part of the link. In this case, "Click here to visit Example."

When the user clicks the link, they will be directed to "<https://www.example.com>".

Opening Links in a New Tab or Window

To make a link open in a new tab or window, you can use the `target="_blank"` attribute.

Example:

```
<a href="https://www.example.com" target="_blank">Visit Example in a new tab</a>
```

- `target="_blank"`: This attribute opens the linked document in a new browser tab or window.

Linking to Other Pages in the Same Website

You can also link to other pages within the same website by using relative URLs. For example, if you have a page named `about.html`, you can link to it like this:

Example:

```
<a href="about.html">About Us</a>
```

This link will take the user to the `about.html` page in the same directory.

Anchor Links (Jump to Specific Sections on the Same Page)

If you want to create links that jump to specific sections within the same page (also known as anchor links), you need to assign an `id` to the target section and reference that `id` in the link.

Example:

```
<!-- Link to a section -->
<a href="#section2">Go to Section 2</a>

<!-- Target section with an id -->
<h2 id="section2">This is Section 2</h2>
<p>Content of Section 2.</p>
```

- `href="#section2"`: The `#section2` part refers to the `id="section2"` of the target element.

When the user clicks the link, they will be taken to the section with the `id="section2"`.

5.2 Navigation Bars

Navigation bars are essential components of a website. They allow users to navigate to different sections or pages of the site. Typically, a navigation bar contains links that are styled to appear horizontally or vertically.

A common way to structure a navigation bar is by using an **unordered list** (`<ul>`) with each link wrapped in a list item (`<li>`).

Basic Navigation Bar Structure

Here's an example of a simple horizontal navigation bar using an unordered list:

Example of a Horizontal Navigation Bar:

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="services.html">Services</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>
```

- `<nav>`: The `<nav>` element is used to define a section of the page that contains navigation links. This is semantic HTML and helps with accessibility.
- `<ul>`: An unordered list is used to structure the links.
- `<li>`: List items that each contain a link.
- `<a href="...">`: Each link takes the user to a different page or section.

Styling the Navigation Bar

To make the navigation bar horizontal, you can use **CSS** to remove the bullets from the list and display the items in a row.

Example CSS for Horizontal Navigation:

```
<style>
  nav ul {
    list-style-type: none; /* Removes bullets */
    padding: 0;
    margin: 0;
```



```

        text-align: center;
    }

    nav ul li {
        display: inline-block; /* Displays the list items in
a row */
        margin: 10px;
    }

    nav ul li a {
        text-decoration: none; /* Removes underline from link
s */
        padding: 10px;
        background-color: #333;
        color: white;
        border-radius: 5px;
    }

    nav ul li a:hover {
        background-color: #555; /* Changes color on hover */
    }
</style>

```

With this CSS, the navigation links will be displayed horizontally, with a background color and hover effect.

Example of Vertical Navigation Bar

For a vertical navigation bar, you can simply stack the list items instead of displaying them in a row.

Example CSS for Vertical Navigation:

```

<style>
    nav ul {
        list-style-type: none; /* Removes bullets */
        padding: 0;
    }

```

```

    }

    nav ul li {
        margin: 10px 0; /* Adds space between the list items
*/
    }

    nav ul li a {
        text-decoration: none;
        padding: 10px;
        background-color: #333;
        color: white;
        display: block; /* Makes the links block-level, so th
ey span the full width */
        border-radius: 5px;
    }

    nav ul li a:hover {
        background-color: #555; /* Changes color on hover */
    }
</style>

```

This will display the links as a vertical navigation menu.

Summary of Chapter 5: Links and Navigation

- **Creating Links:**

- Use the `<a>` tag to create hyperlinks. The `href` attribute specifies the link destination.
- To open a link in a new tab, use `target="_blank"`.
- Anchor links allow navigation within the same page using `id` attributes.

- **Navigation Bars:**

- Navigation bars can be created using an unordered list (`<ul>`) with list items (`<li>`) containing links (`<a>`).

- You can style the navigation bar using CSS to make it horizontal or vertical and add hover effects.

References:

- [HTML Links - Mozilla Developer Network \(MDN\)](#)
- [HTML Navigation - W3Schools](#)

Chapter 6: Images and Multimedia

- **6.1 Adding Images**

- Using the `<img>` tag, understanding attributes like `src` and `alt`.

- **6.2 Embedding Videos**

- Introduction to embedding multimedia content.
-

6.1 Adding Images

Images are a vital part of modern websites. The `<img>` tag is used to add images to a webpage in HTML. An image can be sourced from either local files or online URLs.

Using the `<img>` Tag

The `<img>` tag is an empty tag, meaning it does not have a closing tag. It uses the `src` (source) attribute to specify the image file and the `alt` (alternative text) attribute for accessibility and SEO.

Basic Image Syntax:

```

```

- `src`: The `src` attribute specifies the location of the image file. This can be a local file path or an online URL.
- `alt`: The `alt` attribute provides alternative text for the image. This text is displayed if the image cannot be loaded and helps visually impaired users

understand the content through screen readers.

Example of Adding an Image:

```

```

- If the image is located in the same directory as the HTML file, you can specify the filename (`cat.jpg`).
- If the image is hosted online, you can provide the full URL:

```

```

Additional Image Attributes

- `width` and `height`: You can specify the width and height of an image in pixels (px) or percentage (%).

```

```

- **Responsive Images:** To make images responsive (i.e., adapt to different screen sizes), you can set the width to `100%` and the height to `auto` to maintain the image's aspect ratio:

```

```

- `title`: The `title` attribute is optional and provides additional information about the image when the user hovers over it.

```

```

Image Accessibility

The `alt` attribute is important for accessibility, especially for screen readers used by visually impaired users. Always include a descriptive `alt` attribute to ensure that all users can understand the content of the image.

6.2 Embedding Videos

In addition to images, multimedia content such as videos can greatly enhance user experience on websites. The HTML5 `<video>` tag is used to embed video files in a webpage.

Using the `<video>` Tag

The `<video>` tag allows you to embed a video, and it can include several attributes for better functionality and control.

Basic Video Syntax:

```
<video src="video.mp4" controls>
  Your browser does not support the video tag.
</video>
```

- `src`: Specifies the video file (either local or online).
- `controls`: This attribute adds built-in controls for playing, pausing, and adjusting volume.
- **Fallback Content:** The text "Your browser does not support the video tag." is displayed if the browser does not support the `<video>` tag.

Multiple Video Formats

For broader compatibility across different browsers, it's a good idea to provide multiple video formats. The most common formats are **MP4**, **WebM**, and **OGG**.

Example of Providing Multiple Formats:

```
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
```

```
<source src="video.ogv" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

- **<source>**: This element allows you to specify different video formats. The browser will use the first format it can play.

Other Useful Video Attributes

- **autoplay**: This attribute automatically starts the video as soon as it is ready.

```
<video src="video.mp4" autoplay controls></video>
```

- **loop**: The **loop** attribute makes the video play continuously in a loop.

```
<video src="video.mp4" loop controls></video>
```

- **muted**: The **muted** attribute ensures the video is initially played without sound.

```
<video src="video.mp4" muted controls></video>
```

- **poster**: The **poster** attribute specifies an image to be displayed before the video starts playing. This image will be shown as the video preview.

```
<video src="video.mp4" poster="preview.jpg" controls></video>
```

Embedding Videos from External Sources (e.g., YouTube)

You can also embed videos from external platforms like YouTube by using an `iframe`. The embed code is typically provided by the video hosting platform.

Example of Embedding a YouTube Video:

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/dQw4w9WgXcQ" frameborder="0" allowfullscreen></iframe>
```

>

- `<iframe>`: The `iframe` tag allows embedding external content. Here, it's used to embed a YouTube video.

Summary of Chapter 6: Images and Multimedia

- **Adding Images:**
 - Use the `<img>` tag to add images to your webpage.
 - Essential attributes include `src` (source of the image) and `alt` (alternative text for accessibility).
 - You can also use `width`, `height`, and `title` to further customize the image display.
 - Always include descriptive `alt` text for accessibility purposes.
- **Embedding Videos:**
 - Use the `<video>` tag to embed video files.
 - Provide multiple video formats using the `<source>` tag for cross-browser compatibility.
 - Utilize attributes like `controls`, `autoplay`, `loop`, and `muted` to control video behavior.
 - You can embed external videos (e.g., from YouTube) using an `<iframe>`.

References:

- [HTML Image - Mozilla Developer Network \(MDN\)](#)
- [HTML Video - Mozilla Developer Network \(MDN\)](#)
- [HTML Video - W3Schools](#)

Chapter 7: Tables

- **7.1 Creating Tables**
 - Using `<table>`, `<tr>`, `<th>`, and `<td>` to structure tabular data.

- **7.2 Table Attributes**

- Understanding attributes like `colspan` and `rowspan`.

7.1 Creating Tables

Tables are useful for displaying tabular data on a webpage. HTML provides several tags to structure and organize table content.

Basic Table Structure

A table in HTML is created using the `<table>` tag. A table consists of rows (`<tr>`), headers (`<th>`), and data cells (`<td>`).

Here's the basic structure of an HTML table:

Table Structure:

```
<table>
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
    <th>Header 3</th>
  </tr>
  <tr>
    <td>Data 1</td>
    <td>Data 2</td>
    <td>Data 3</td>
  </tr>
  <tr>
    <td>Data 4</td>
    <td>Data 5</td>
    <td>Data 6</td>
  </tr>
</table>
```

- `<table>`: Defines the table.

- `<tr>` : Defines a table row.
- `<th>` : Defines a table header. By default, the text in `<th>` elements is bold and centered.
- `<td>` : Defines a table data cell that holds actual data.

In this example, the table has three columns and two rows of data (excluding the header row).

Example Table:

Header 1	Header 2	Header 3
Data 1	Data 2	Data 3
Data 4	Data 5	Data 6

7.2 Table Attributes

HTML tables have several attributes that can enhance their appearance and functionality. Two important attributes are `colspan` and `rowspan`.

`colspan` Attribute

The `colspan` attribute allows a cell to span across multiple columns. It is used within a `<td>` or `<th>` element to specify how many columns the cell should span.

Example of `colspan` :

```
<table>
  <tr>
    <th colspan="3">Header Spanning 3 Columns</th>
  </tr>
  <tr>
    <td>Data 1</td>
    <td>Data 2</td>
    <td>Data 3</td>
  </tr>
</table>
```

- The header in the first row will span across three columns because of `colspan="3"`.

rowspan Attribute

The `rowspan` attribute allows a cell to span across multiple rows. It is used within a `<td>` or `<th>` element to specify how many rows the cell should span.

Example of `rowspan` :

```
<table>
  <tr>
    <th rowspan="2">Header Spanning 2 Rows</th>
    <td>Data 1</td>
    <td>Data 2</td>
  </tr>
  <tr>
    <td>Data 3</td>
    <td>Data 4</td>
  </tr>
</table>
```

- The header in the first column will span across two rows because of `rowspan="2"`.

Additional Table Attributes

In addition to `colspan` and `rowspan`, there are other attributes that can be used to customize tables:

- **border** : The `border` attribute adds a border around the table. (Note: CSS is often preferred for styling tables instead of the `border` attribute.)

```
<table border="1">
```

- **width** and **height** : You can specify the width and height of a table and its cells.

```
<table width="500" height="300">
```

- **align**: The `align` attribute can be used to align the content of the table, although it is now considered deprecated in favor of CSS.

```
<table align="center">
```

- **caption**: You can add a title or description for the table using the `<caption>` tag.

```
<table>
  <caption>Employee Data</caption>
  <tr><td>Data</td></tr>
</table>
```

Summary of Chapter 7: Tables

- **Creating Tables:**
 - Use the `<table>` tag to create a table.
 - Use `<tr>` for rows, `<th>` for headers, and `<td>` for data cells.
 - You can span columns with `colspan` and rows with `rowspan` to create more complex tables.
- **Table Attributes:**
 - `colspan`: Allows a cell to span across multiple columns.
 - `rowspan`: Allows a cell to span across multiple rows.
 - Additional attributes like `border`, `width`, and `height` help define the table's appearance.

References:

- [HTML Tables - Mozilla Developer Network \(MDN\)](#)
- [HTML Tables - W3Schools](#)

Chapter 8: Forms and User Input

- **8.1 Creating Forms**

- Using the `<form>` tag to collect user input.

- **8.2 Input Types**

- Different types of inputs (text, radio buttons, checkboxes).

8.1 Creating Forms

Forms are essential for collecting user input in web applications. HTML forms allow users to submit data (e.g., text, selections, and files) to a server for processing. The `<form>` tag is used to define an HTML form, and it contains various form elements like text fields, radio buttons, checkboxes, and submit buttons.

Basic Form Structure

A basic form uses the `<form>` tag, which can have attributes such as `action`, `method`, and `name`. The `action` attribute specifies the URL where the form data will be sent, and the `method` attribute determines how the data will be sent (usually `GET` or `POST`).

Basic Form Example:

```
<form action="/submit_form" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username"><br><br>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password"><br>
<br>

  <input type="submit" value="Submit">
</form>
```

- `<form>` : Defines the form.
- `action="/submit_form"` : Specifies the URL to which the form data will be sent when the form is submitted.
- `method="POST"` : Specifies the HTTP method to be used when submitting the form data. `POST` sends data in the body of the request, while `GET` appends the data to the URL.
- `<label>` : Used to define labels for form elements. It's important for accessibility and improves user interaction.
- `<input>` : Defines an input field. The `type` attribute specifies the type of input (e.g., text, password, submit button).
- `<input type="submit">` : A button that submits the form.

Form Submission

When the form is submitted, the browser sends the form data to the server specified in the `action` attribute using the `method` defined. The data can be processed by a server-side script (like PHP, Node.js, or Python).

8.2 Input Types

Forms support various types of inputs that allow users to enter different kinds of data. The `<input>` tag is used for most form elements, and its `type` attribute specifies the kind of input field. Here are some common input types:

1. Text Input (`<input type="text">`)

Used for single-line text input, like a username or search field.

Example:

```
<label for="name">Name:</label>
<input type="text" id="name" name="name">
```

2. Password Input (`<input type="password">`)

Similar to a text input field, but the entered text is hidden (useful for passwords).

Example:

```
<label for="password">Password:</label>
<input type="password" id="password" name="password">
```

3. Radio Buttons (`<input type="radio">`)

Radio buttons allow users to select one option from a group. All radio buttons with the same `name` belong to the same group, and only one can be selected at a time.

Example:

```
<p>Choose your favorite color:</p>
<input type="radio" id="red" name="color" value="red">
<label for="red">Red</label><br>
<input type="radio" id="blue" name="color" value="blue">
<label for="blue">Blue</label><br>
<input type="radio" id="green" name="color" value="green">
<label for="green">Green</label>
```

- `name="color"` : All radio buttons with the same `name` attribute belong to the same group.
- `value="red"` : The value that is sent when the form is submitted.

4. Checkboxes (`<input type="checkbox">`)

Checkboxes allow users to select one or more options. Unlike radio buttons, checkboxes are independent and can be selected simultaneously.

Example:

```
<p>Select your hobbies:</p>
<input type="checkbox" id="reading" name="hobby" value="reading">
<label for="reading">Reading</label><br>
<input type="checkbox" id="sports" name="hobby" value="sport
```

```
s">
<label for="sports">Sports</label><br>
<input type="checkbox" id="music" name="hobby" value="music">
<label for="music">Music</label>
```

- `name="hobby"` : Since the checkboxes are independent, they can have the same `name` if you want them to be part of the same group.
- `value="reading"` : The value that is sent when the checkbox is checked.

5. Submit Button (`<input type="submit">`)

A submit button sends the form data to the server.

Example:

```
<input type="submit" value="Submit">
```

- `value="Submit"` : Specifies the text on the button.

6. Email Input (`<input type="email">`)

The email input type validates that the entered text is in the format of an email address (e.g., `user@example.com`).

Example:

```
<label for="email">Email:</label>
<input type="email" id="email" name="email">
```

7. Number Input (`<input type="number">`)

The number input type allows users to enter only numeric values. You can also specify a range of acceptable values.

Example:

```
<label for="age">Age:</label>
<input type="number" id="age" name="age" min="18" max="100">
```

- `min="18"` and `max="100"`: Specify the acceptable range of numbers.

8. Date Input (`<input type="date">`)

This input type allows users to select a date from a calendar popup.

Example:

```
<label for="birthday">Birthday:</label>
<input type="date" id="birthday" name="birthday">
```

9. Textarea (`<textarea>`)

The `<textarea>` tag is used for multi-line text input, such as comments or messages. It has `rows` and `cols` attributes to control its size.

Example:

```
<label for="message">Message:</label><br>
<textarea id="message" name="message" rows="4" cols="50"></te
xtarea>
```

- `rows="4"`: Specifies the number of visible text lines.
- `cols="50"`: Specifies the width of the text area.

Summary of Chapter 8: Forms and User Input

- **Creating Forms:**

- Use the `<form>` tag to create forms that collect user data.
- The `action` attribute defines where to send the form data, and the `method` attribute specifies how the data will be sent.

- Input elements like `<input>`, `<textarea>`, and `<select>` are used to collect various types of data.
- **Input Types:**
 - **Text Input:** `<input type="text">` for single-line text.
 - **Password Input:** `<input type="password">` hides text input.
 - **Radio Buttons:** `<input type="radio">` for selecting one option from a group.
 - **Checkboxes:** `<input type="checkbox">` for multiple selections.
 - **Submit Button:** `<input type="submit">` to submit the form.
 - Other types include **email**, **number**, **date**, and **textarea** for multi-line input.

References:

- [HTML Forms - Mozilla Developer Network \(MDN\)](#)
- [HTML Forms - W3Schools](#)

Chapter 9: Semantic HTML

- **9.1 Understanding Semantic Elements**
 - Importance of semantic tags like `<header>`, `<footer>`, `<article>`, etc.

9.1 Understanding Semantic Elements

Semantic HTML refers to the use of HTML tags that have meaningful names, making it easier for both humans and machines (like search engines and screen readers) to understand the structure and content of a webpage. Using semantic elements helps improve accessibility, SEO (Search Engine Optimization), and maintainability of web pages.

Semantic elements describe their meaning in a way that is understandable to both the browser and the developer, as opposed to non-semantic elements (like `<div>` and `<span>`) which have no inherent meaning and are often used purely for styling or layout purposes.

Why Semantic HTML is Important

1. **Improved Accessibility:** Semantic tags make it easier for screen readers and other assistive technologies to understand the page structure and provide better user experiences for people with disabilities.
 2. **Better SEO:** Search engines like Google can more easily crawl and index content structured with semantic HTML, improving page rankings.
 3. **Easier Maintenance:** Clear and meaningful HTML makes code more readable and easier to maintain or modify.
 4. **Consistent Layouts:** Semantic tags help provide clear distinctions between sections of a webpage, leading to more consistent designs.
-

Common Semantic HTML Elements

1. `<header>`

The `<header>` element defines the top section of a page or a section. It typically contains headings, navigation links, logos, or introductory content.

Example:

```
<header>
  <h1>Welcome to My Website</h1>
  <nav>
    <ul>
      <li><a href="#home">Home</a></li>
      <li><a href="#about">About</a></li>
      <li><a href="#services">Services</a></li>
    </ul>
  </nav>
</header>
```

- The `<header>` tag often contains introductory or navigational content.

2. `<footer>`

The `<footer>` element represents the bottom section of a page or a section. It is often used for copyright information, contact details, or links to privacy policies.

Example:

```
<footer>
  <p>&copy; 2025 My Website. All rights reserved.</p>
  <ul>
    <li><a href="#privacy">Privacy Policy</a></li>
    <li><a href="#terms">Terms of Service</a></li>
  </ul>
</footer>
```

- The `<footer>` tag contains footer-related information, such as legal and administrative details.

3. `<article>`

The `<article>` element defines independent, self-contained content that can be reused or distributed independently (like blog posts, news articles, or forum posts). It typically has its own heading and body content.

Example:

```
<article>
  <h2>Breaking News: Web Development Trends in 2025</h2>
  <p>Web development continues to evolve with new technologies and frameworks...</p>
</article>
```

- The `<article>` tag is used to mark up content that can stand alone, like a blog post or news article.

4. `<section>`

The `<section>` element defines a section of content within a webpage. It is used to group related content, and each section typically has a heading (e.g., `<h2>`).

Example:

```
<section>
  <h2>Our Services</h2>
  <p>We offer web design, development, and marketing services...</p>
</section>
```

- The `<section>` tag is used for sections of related content within the page, like different services or topics.

5. `<nav>`

The `<nav>` element is used to define navigation links. It groups the links that allow users to navigate to different sections of a website or across websites.

Example:

```
<nav>
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>
```

- The `<nav>` tag helps define a block of links used for navigation.

6. `<aside>`

The `<aside>` element is used for content that is tangentially related to the main content. It could be a sidebar, pull quotes, or related links.

Example:

```
<aside>
  <h3>Related Articles</h3>
```

```

    <ul>
      <li><a href="#">How to Learn HTML</a></li>
      <li><a href="#">CSS Tips and Tricks</a></li>
    </ul>
  </aside>

```

- The `<aside>` tag is used for content like sidebars or supplementary material.

7. `<main>`

The `<main>` element represents the primary content of a document. It excludes content that is repeated across pages (like headers, footers, and navigation). There should only be one `<main>` element per page.

Example:

```

<main>
  <h2>Welcome to My Web Development Portfolio</h2>
  <p>Here are some of the projects I've worked on...</p>
</main>

```

- The `<main>` tag is used to encapsulate the core content of a page.

8. `<figure>` and `<figcaption>`

The `<figure>` element is used to group media content (like images, videos, or diagrams), and the `<figcaption>` element provides a caption or description for that content.

Example:

```

<figure>
  
  <figcaption>Example of a responsive website design.</figcaption>
</figure>

```

- The `<figure>` tag is used for media content, and the `<figcaption>` tag provides a caption.

9. `<mark>`

The `<mark>` element is used to highlight parts of the text. It is typically used to highlight keywords or search results.

Example:

```
<p>The <mark>HTML5</mark> specification includes many new elements.</p>
```

- The `<mark>` tag highlights content within the page.

Benefits of Using Semantic HTML

- **Accessibility:** Semantic HTML improves the experience for users with disabilities, as screen readers can interpret the meaning of the tags and navigate the page more efficiently.
- **SEO:** Search engines use semantic HTML to better understand the structure of a page and improve rankings.
- **Code Readability:** Using meaningful tags makes your code more readable and easier to maintain for other developers.
- **Consistent Layouts:** Semantic tags help define the structure of a webpage, leading to more organized and maintainable layouts.

Summary of Chapter 9: Semantic HTML

- **Semantic Tags:** Tags like `<header>`, `<footer>`, `<article>`, `<section>`, and `<nav>` provide clear meaning to different parts of a webpage.
- **Improved Accessibility:** Semantic elements make it easier for screen readers to understand and navigate the content.
- **Better SEO:** Search engines can better index content when semantic tags are used.

- **Code Organization:** Using meaningful tags improves the readability, maintainability, and structure of the code.

By using semantic HTML, developers create content that is both machine-readable and human-friendly, which contributes to a better user experience and improved search engine rankings.

References:

- [HTML5 Semantic Elements - Mozilla Developer Network \(MDN\)](#)
- [Semantic HTML - W3Schools](#)

Chapter 10: Responsive Design Basics

- **10.1 Introduction to Responsive Web Design**
 - Basic principles of making websites responsive using media queries.

10.1 Introduction to Responsive Web Design

Responsive Web Design (RWD) is an approach to web design aimed at creating web pages that look and function well on devices of all sizes, from desktops to smartphones. The main principle behind responsive design is that the layout and content should adapt to the screen size and resolution, ensuring an optimal viewing experience across a wide range of devices.

Why Responsive Web Design is Important

- **User Experience:** With the increase in mobile browsing, it's important for websites to be easily accessible and user-friendly across all devices.
- **SEO:** Google favors mobile-friendly websites and ranks them higher in search results.
- **Cost Efficiency:** A responsive design allows you to maintain one codebase for multiple devices, rather than creating separate websites for desktop and mobile.

Key Principles of Responsive Web Design

1. **Fluid Layouts:** Use relative units like percentages, `vw` (viewport width), and `vh` (viewport height) for widths and heights, rather than fixed pixel values. This allows elements to resize fluidly based on the screen size.

Example:

```
.container {  
    width: 100%; /* The container will take up 100% of the  
    available width */  
}
```

2. **Flexible Images:** Images should be set to scale according to the available width. This ensures that images look good on all screen sizes without being stretched or compressed.

Example:

```
img {  
    max-width: 100%;  
    height: auto; /* This ensures that the image scales pr  
    oportionally */  
}
```

3. **Media Queries:** Media queries are the core of responsive web design. They allow you to apply different styles based on the screen's characteristics, such as width, height, orientation, and resolution.

Example of a basic media query:

```
/* Default styles for larger screens */  
body {  
    font-size: 18px;  
}  
  
/* Styles for devices with a max-width of 600px (e.g., mob  
ile phones) */
```



```
@media screen and (max-width: 600px) {  
    body {  
        font-size: 14px; /* Adjust font size for small screens */  
    }  
}
```

- **@media screen** : Defines the media type (screen refers to computer screens, tablets, and phones).
- **(max-width: 600px)** : The condition that triggers the media query. It applies the defined styles if the screen width is less than or equal to 600px.

Responsive Design Techniques

1. Viewport Meta Tag

The viewport meta tag is used to control the layout on mobile devices. It defines the width of the viewport (the visible area of the webpage) and sets the initial zoom level.

Example:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- **width=device-width** : Ensures the viewport is set to the device's width, not the default width of 980px (which older desktop-oriented sites use).
- **initial-scale=1** : Sets the initial zoom level to 1, ensuring that the page is not zoomed in or out on load.

2. Mobile First Design

Mobile-first design is a strategy where you start by designing for small screens (e.g., smartphones) and gradually add features for larger screens using media queries. This approach ensures that your website will be optimized for the smallest screen size and scale up for larger screens.

Example:

```
/* Mobile-first styles (for screens 600px or less) */
body {
    font-size: 14px;
}

@media screen and (min-width: 601px) {
    /* Styles for screens larger than 600px */
    body {
        font-size: 18px;
    }
}

@media screen and (min-width: 1024px) {
    /* Styles for screens larger than 1024px (tablet and desk
top) */
    body {
        font-size: 20px;
    }
}
```

3. Fluid Grid Layouts

Instead of using fixed widths (in pixels), use flexible grid layouts. This can be done by setting the width of columns and containers in percentages rather than pixels.

Example:

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr; /* Creates 3 equal-wi
dth columns */
    gap: 10px;
}
```

```
@media screen and (max-width: 600px) {
  .container {
    grid-template-columns: 1fr; /* Stacks columns vertically on smaller screens */
  }
}
```

- **1fr** : Denotes a fraction of the available space, allowing for flexible, proportional layouts.
- **gap** : Defines the space between grid items.

4. Responsive Typography

Responsive typography ensures that font sizes adjust based on the screen size. You can use relative units like **em**, **rem**, or **vw** to make the text more flexible.

Example:

```
body {
  font-size: 2vw; /* Font size adjusts based on the viewport width */
}

@media screen and (max-width: 600px) {
  body {
    font-size: 4vw; /* Increase font size on smaller screens */
  }
}
```

- **vw (viewport width)**: The font size will be a percentage of the viewport's width.

Common Media Query Breakpoints

When creating a responsive website, you typically target certain breakpoints where the layout needs adjustment. These breakpoints correspond to common screen widths for various devices.

Here are some common breakpoints:

- **Mobile devices (portrait):** `max-width: 600px`
- **Tablets (portrait):** `min-width: 601px` and `max-width: 1024px`
- **Larger tablets and small laptops:** `min-width: 1025px` and `max-width: 1200px`
- **Desktops:** `min-width: 1201px`

Example of Media Query Breakpoints:

```
/* Mobile devices */
@media screen and (max-width: 600px) {
    /* Mobile-specific styles */
}

/* Tablets */
@media screen and (min-width: 601px) and (max-width: 1024px)
{
    /* Tablet-specific styles */
}

/* Laptops and Desktops */
@media screen and (min-width: 1025px) {
    /* Desktop-specific styles */
}
```

Summary of Chapter 10: Responsive Design Basics

- **Responsive Web Design (RWD)** ensures that web content looks good on all devices, from smartphones to desktops.
- **Key Techniques:**

- **Fluid Layouts:** Use relative units (like percentages, `vw`, `vh`) instead of fixed pixel values.
- **Flexible Images:** Images scale properly based on the available width.
- **Media Queries:** Use media queries to apply different styles based on screen size and device characteristics.
- **Viewport Meta Tag:** Ensures proper scaling on mobile devices.
- **Mobile-First Design:** Start designing for smaller screens and progressively enhance for larger screens.
- **Responsive Typography:** Adjust font sizes based on screen size for better readability.

By incorporating responsive web design practices, you can create websites that provide an optimal user experience across a variety of devices, improving accessibility, user engagement, and SEO.

References:

- [Responsive Web Design - Mozilla Developer Network \(MDN\)](#)
- [Responsive Web Design - W3Schools](#)

Chapter 11: Practical Project

- **11.1 Building a Simple Web Page**
 - Applying what you've learned by creating a complete web page from scratch.